Bank Campaign Predictor – AI Portfolio Report

Introduction to AI (AIN5301EN)

Task 1 – Problem Definition

1. Web app idea

My app is a **Bank Campaign Predictor**. When a bank runs a phone campaign for term deposits, most customers say no, so staff spend a lot of time on calls that lead nowhere. The app predicts which customers are more likely to say yes, so the bank can call them first. It is my own idea, but it is close to the Customer Churn Predictor and Loan Approval Checker from the brief.

2. The MVP

- **What it does:** an employee enters a customer's details in a simple form and gets a yes/no prediction with a probability. Each prediction is saved in a database.
- **Who uses it:** marketing and call centre staff who plan the campaign.
- **Problem it solves:** fewer wasted calls and fewer customers bothered by calls they do not want.

3. AI component

The model classifies a customer as likely to subscribe (yes) or not (no). Simple rules are not enough here, because the answer depends on many factors at once, such as job, age, previous contact and month. A model can learn these combinations from 41,188 past customers and can be retrained when new data arrives.

4. Inputs and outputs

Input	Output
Age, job, marital status, education	Yes / No
Credit default, housing loan, personal loan	Probability of yes (0–100%)
Contact type, month, day of week	Call / lower priority

Input	Output
Contacts in this and previous campaigns, previous result	

5. Mock data

Since the MVP starts without real users, below is an example of the data the system would store. The values are my own assumptions, based on what a bank keeps in its customer records and past campaign results.

CustomerID	Age	Job	Housing loan	Previous result	Contacts	Subscribed
1	28	student	no	success	1	yes
2	45	blue-collar	yes	nonexistent	5	no
3	63	retired	no	failure	2	yes
4	36	admin.	yes	nonexistent	3	no

A few mock rows are not enough to train a model, so for training I used a public dataset (Task 2).

Task 2 – Database Design

Dataset

I used the **Bank Marketing dataset** (`bank-additional-full.csv`) from Kaggle / UCI. It contains real phone campaign data from a Portuguese bank between 2008 and 2010: 41,188 customers, 21 columns and no missing values. Only 11.3% of the customers said yes. I chose it because it fits my idea and most of its columns are easy to understand and use in a form.

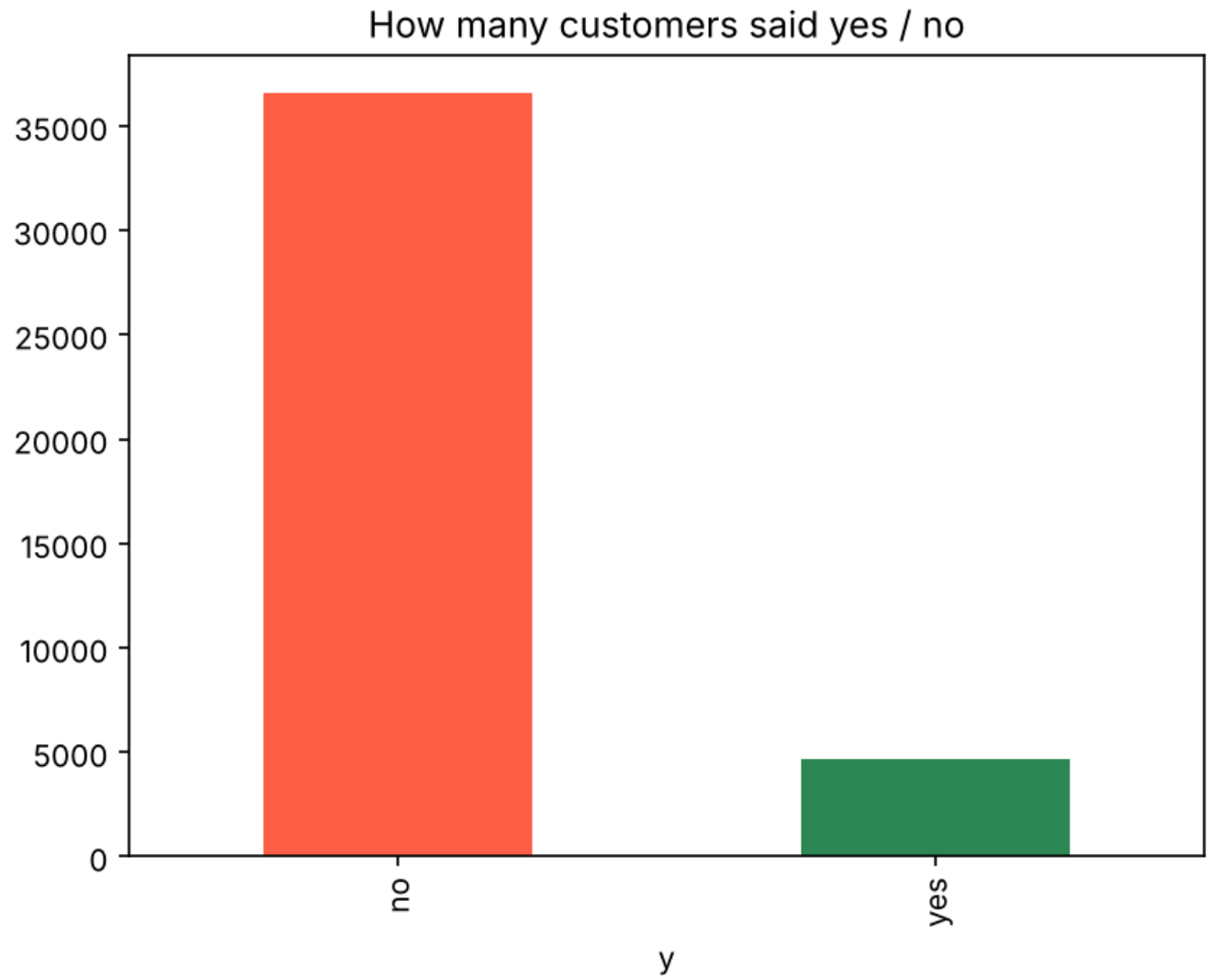


Figure 1: The dataset is imbalanced – about 89% of customers said no.

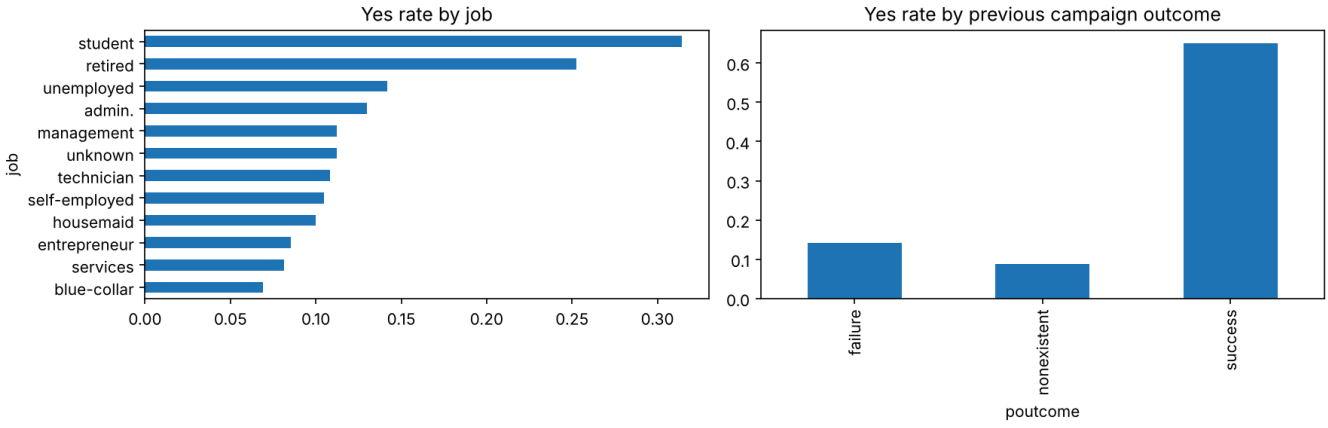


Figure 2: Students, retired people and customers with a successful previous campaign say yes more often.

Database

I used **SQLite**, since it is a single file (`bank.db`), comes with Python and needs no server. It has two tables.

`customers` holds the historical data used for training:

Column	Type	Example
age	INTEGER	56
job, marital, education	TEXT	housemaid, married, basic.4y
default, housing, loan	TEXT	no / yes / unknown
contact, month, day_of_week	TEXT	telephone, may, mon
campaign, pdays, previous	INTEGER	1, 999, 0
poutcome	TEXT	nonexistent / failure / success
duration + 5 economic columns	INTEGER / REAL	(not used by the model)
y	TEXT	yes / no

`predictions` is created by the app and stores every prediction:

Column	Type	Example
id	INTEGER (primary key)	1
created_at	TEXT	2026-11-07 09:52
customer_name	TEXT	Maria Test
age, job, marital, education, poutcome, campaign	INTEGER / TEXT	35, admin., married, …
probability	REAL	0.494
prediction	TEXT	no

The notebook loads the CSV into `customers` and reads it back with SQL to train the model.

Task 3 – Implementing the AI Technique

All the code is in `bank_campaign_model.ipynb`. The best model was a **Random Forest** (ROC AUC 0.782), which finds 58% of the customers who say yes.

Preprocessing

1. Removed `duration` (length of the call). It is only known after the call ends, so keeping it would let the model "cheat".
2. Removed the 5 economic indicators, because a bank employee would not type them into a form.
3. Turned `pdays = 999` (never contacted) into a simple 0/1 column, `contacted_before`.
4. One-hot encoded the text columns and scaled the numeric ones.
5. Split the data 80/20 into training (32,950) and test (8,238) sets, keeping the same yes/no ratio in both.

```
drop_cols = ["duration", "emp_var_rate", "cons_price_idx", "cons_conf_idx",
"euribor3m", "nr_employed"]
data = df.drop(columns=drop_cols)
data["contacted_before"] = (data["pdays"] != 999).astype(int)

preprocess = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
    ("num", StandardScaler(), num_cols),
])
```

Algorithm choice and training

I compared three classifiers: **Logistic Regression** as a simple baseline, a **Decision Tree** because it is easy to interpret, and a **Random Forest** (200 trees), which is usually more accurate. As only 11% of customers say yes, I used `class_weight="balanced"` so the models do not ignore that class.

```
pipe = Pipeline([("prep", preprocess),
                 ("model", RandomForestClassifier(n_estimators=200,
max_depth=10,
                                                  class_weight="balanced",
random_state=42))])
pipe.fit(X_train, y_train)
```

Results (test set, 8,238 customers)

Model	Accuracy	Precision (yes)	Recall (yes)	F1 (yes)	ROC AUC
Logistic Regression	0.797	0.297	0.587	0.394	0.765
Decision Tree	0.870	0.422	0.412	0.417	0.728
Random Forest	0.840	0.365	0.577	**0.447**	**0.782**
Baseline (always "no")	0.887	0	0	0	–

I chose the Random Forest because it had the best F1 and ROC AUC. The Decision Tree has higher accuracy, but it misses more of the yes customers, which is what the app is for. The final model is saved as `model.pkl` and loaded by the app.

Task 4 – Applying AI to Solve the Problem

How the model works

A Random Forest is a group of 200 decision trees, each trained on a slightly different random part of the data. For a new customer every tree votes yes or no, and the share of yes votes becomes the probability. At 50% or above, the app says "likely yes".

The features that mattered most were whether the customer had been contacted before, whether the previous campaign was a success, the contact type and age.

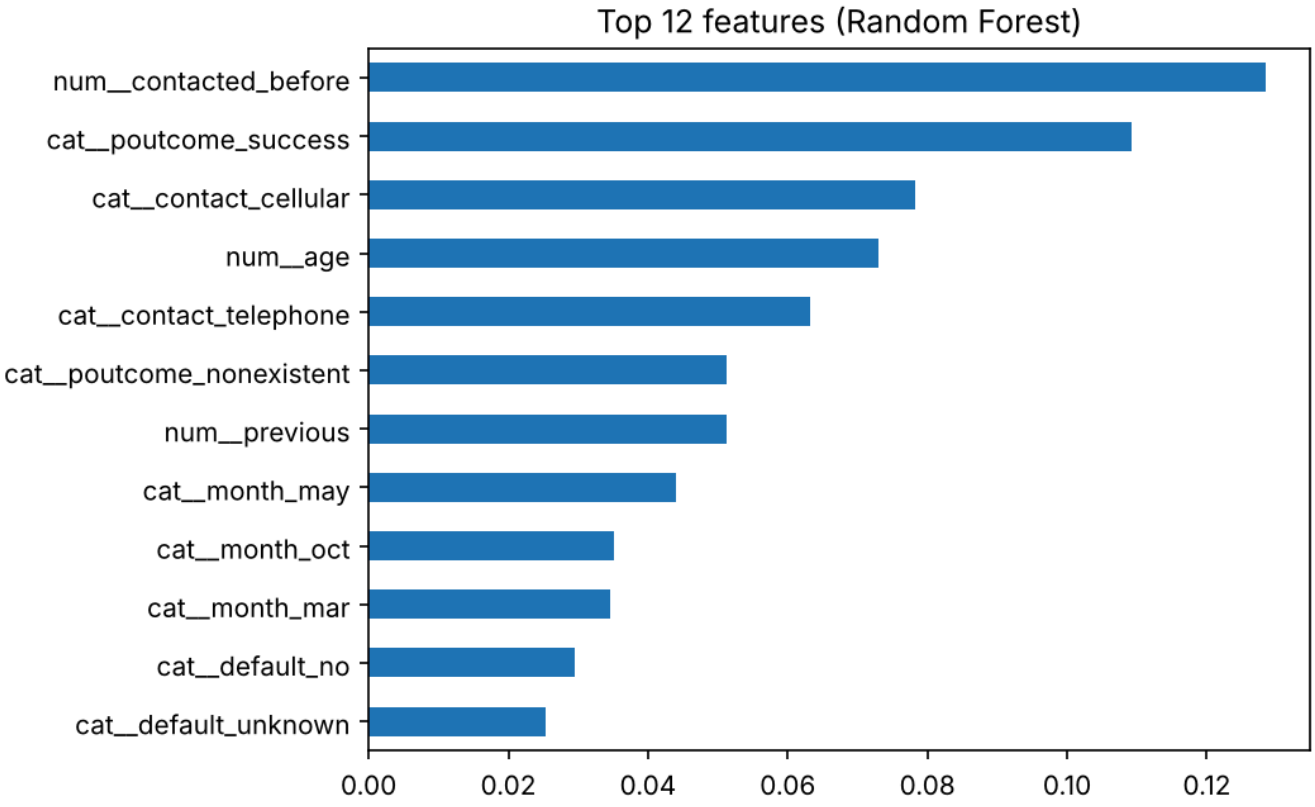


Figure 3: The 12 most important features of the Random Forest model.

Why I chose this technique

The question has two possible answers, so it is a classification problem. Random Forest handles a mix of text and numeric columns well, needs little tuning, gave the best results of the three models, and shows which features matter, which makes the result easier to explain to a bank.

How the database connects to the AI

1. The dataset is stored in the `customers` table of `bank.db`.
2. The notebook reads this table, trains the model and saves it as `model.pkl`.
3. The Streamlit app loads `model.pkl` and sends the form data to it when the user clicks Predict.
4. The result is shown and saved in the `predictions` table, which the user can see in the "Saved predictions" tab.

In a real version, new campaign results could be added to the database and the model retrained regularly.

Example inputs and outputs

Input (customer)	Probability of yes	Output
30, student, single, university degree, no loans, cellular, October, 1 contact, previous campaign **success**	93%	Yes – call this customer
45, blue-collar, married, basic 9y, housing + personal loan, telephone, May, 6 contacts, never contacted before	22%	No – lower priority
35, admin., married, university degree, no loans, cellular, May, 1 contact, never contacted before (app default)	49%	No – lower priority

These results make sense: a customer who said yes before is very likely to say yes again, while someone already called six times without success is unlikely to change their mind.

Task 5 – Evaluation

Accuracy

The Random Forest reached **84.0% accuracy**. That is actually lower than a model that always says "no" (88.7%), but such a model never finds a single interested customer. For imbalanced data like this, recall, precision and F1 tell much more.

Confusion matrix on the test set:

	Predicted no	Predicted yes
Actually no	6,381	929
Actually yes	393	535

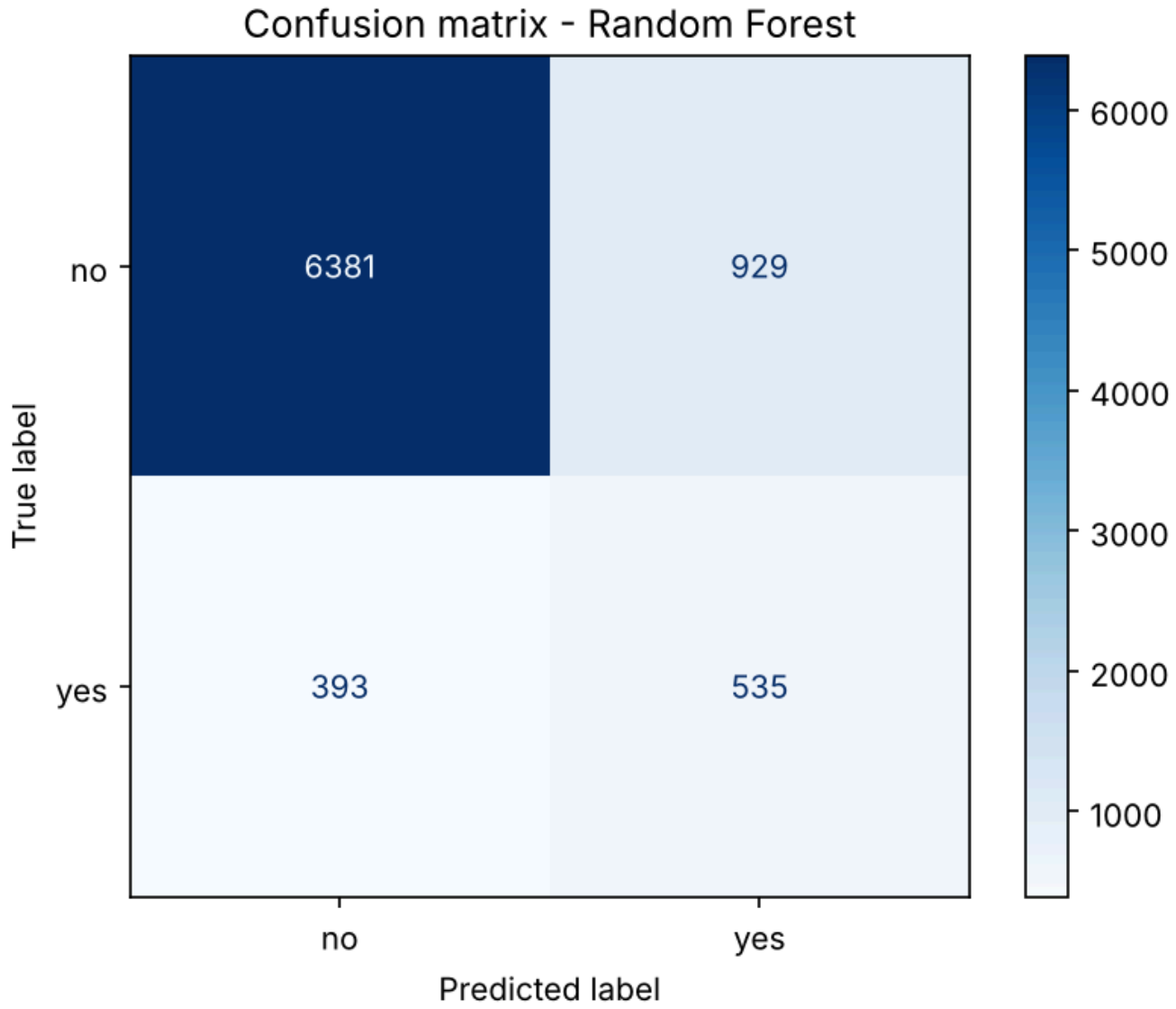


Figure 4: Confusion matrix of the Random Forest on the test set (output of the notebook).

- **Recall 57.7%:** it finds 535 of the 928 customers who said yes.
- **Precision 36.5%:** about 1 in 3 recommended customers says yes, compared to about 1 in 9 when calling at random. So calls become roughly three times more effective.
- **ROC AUC 0.782:** clearly better than random guessing (0.5).

Performance

Predicting all 8,238 test customers takes about 0.1 seconds, and training takes a few seconds on a normal laptop. In the app, the result appears instantly. The model file is 2.7 MB.

Limitations

- **Old data from one bank** (Portugal, 2008–2010), so results may differ for other banks or today.
- **No calls in January and February.** For these months the app averages the prediction over the other ten months and tells the user.
- **Low precision:** about two in three "yes" predictions are still wrong. Calls are cheap, so this is acceptable, but not ideal.
- **Fewer features:** removing `duration` and the economic indicators makes the app realistic but the model slightly weaker.
- **Prototype only:** SQLite and Streamlit are fine for testing, but not for many users, and there is no login.

Improvements

- Tune the model with GridSearchCV and try others, such as XGBoost.
- Adjust the 50% threshold to match how many calls the bank can make.
- Add the economic indicators automatically from a public source.
- Retrain the model regularly with new campaign results.
- Allow uploading a CSV of customers and return a sorted call list.

Use of Generative AI

This assessment is Category C, so Generative AI was used as part of the work. I used **Claude (Anthropic)** as follows:

Part of the work	Use of AI	My contribution
Idea and dataset	Compared my two ideas and suggested the dataset	Proposed the bank-related ideas and chose the final one
Code (notebook, app, database)	Generated the code	Ran and tested everything, solved the installation problems on my laptop, identified the January/February issue and rejected the first fix

Part of the work	Use of AI	My contribution
Report	Drafted the text of Tasks 1–6, based on my answers for the reflection	Provided the content of the reflection, reviewed the text and requested changes

My evaluation of the AI output: The AI was very useful for the coding part, as the notebook and the application worked with only small changes, and I learned a lot by testing them. However, the output was not perfect. The application contained a mistake with the months, which I only found because I tested my own customer with January, and I did not accept the first solution that was suggested. In addition, the code did not run immediately on my laptop, so I had to solve the installation problems myself. My main conclusion is that AI output should not be trusted without running and testing it.

Note: this section was drafted with Claude based on what I actually did, and checked by me.

Task 6 – Portfolio Reflection

Development process. I began by reviewing the suggested ideas in the brief, such as the Student Performance Predictor and the Spam Message Detector. Although they were suitable, I wanted to work on something related to banking and closer to a real business problem, so I developed my own ideas. I considered two options, a customer campaign predictor and a fraud detection system, and finally chose the campaign predictor because the data was easier to understand and could be used directly in a web form. After that, I selected the dataset, trained the model in a Jupyter notebook, built the web application with Streamlit, tested it and wrote this report.

Challenges. The most difficult part for me was setting up the environment. Firstly, pip returned a "Permission denied" error in Git Bash, and then pandas could not be loaded because of a DLL error, which caused the application to show a blank page. I had to reinstall pandas, and the first run took a long time to load. Overall, this stage took more time than the AI part itself. A second challenge appeared during testing, when I entered a customer with January as the contact month. I discovered that the dataset contains no calls in January or February, which meant that the model was effectively guessing for these months. The first solution was to remove these months from the form, but I did not agree with this, since a real bank could contact customers in January. Instead, the application now calculates the average prediction across the other months and informs the user.

What I learned. Before this project I had not used Jupyter notebooks or Streamlit. I now understand that the notebook is where the data is cleaned and the model is trained and evaluated, while the Streamlit application is the part the user interacts with, which loads the saved model and makes predictions. It was useful to see how these two parts are connected through the model file and the database.

Working with AI. The AI tool was helpful, especially for writing the code and explaining concepts step by step. However, in some cases searching online by myself was more helpful, for example when I was trying to understand certain errors and concepts. I also learned that AI output cannot simply be copied and trusted. The code did not work immediately on my laptop, and the issue with the months was only identified because I tested the application myself.

What I would do differently. Next time I would prepare the environment more carefully from the beginning. I had Python 3.14 installed on my laptop, which is a very recent version, and I believe a more stable version such as 3.12 would have saved a lot of time on installation problems. I would also test the application with less common inputs at an earlier stage, since this is how I found the January problem.

Future use. I believe this experience will be useful for data analysis in the future. It also helped me develop a more business-oriented way of thinking, because the goal was not only to achieve high accuracy, but to help the bank decide which customers are worth contacting.

References

- Moro, S., Cortez, P. and Rita, P. (2014) 'A data-driven approach to predict the success of bank telemarketing', *Decision Support Systems*, 62, pp. 22–31.
- Bank Marketing dataset, UCI Machine Learning Repository / Kaggle: archive.ics.uci.edu/dataset/222/bank+marketing
- scikit-learn documentation: scikit-learn.org
- Streamlit documentation: docs.streamlit.io
- Anthropic (2026) Claude [Large language model]. Used for code and draft text, see "Use of Generative AI".
- GitHub repository: [add your link here]